BUSINESS ACADEMY AARHUS

Exercises. ITA-ØK. Week 8. LM Studio.

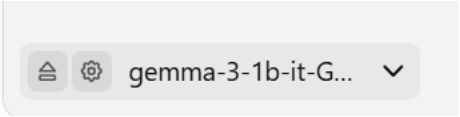
Sila, April 8th, 2026.



Exercise 1.

Download LM Studio.

Try it out with a reasonable sized model.
E.g. a Gemma model.



hello there what are you called ?

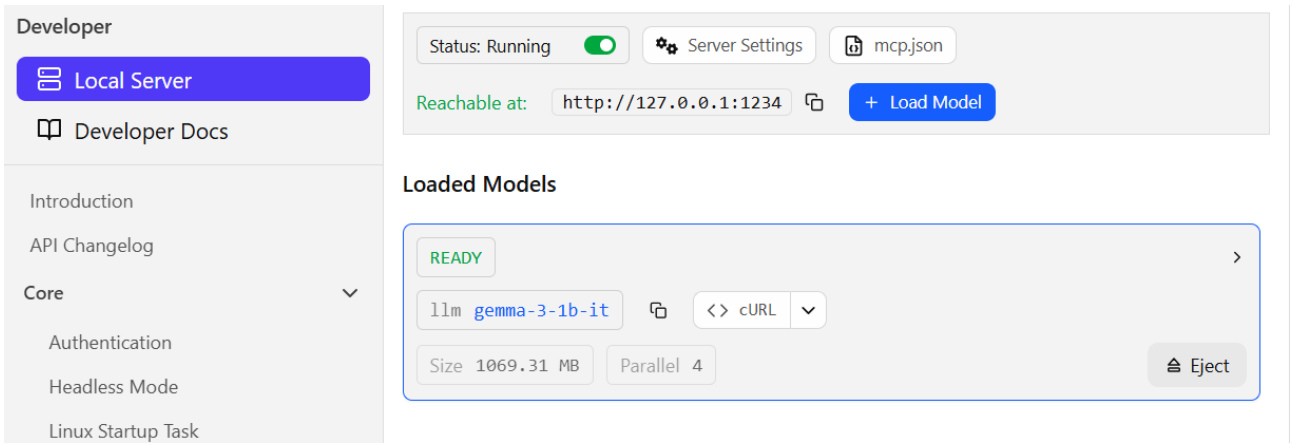
gemma-3-1b-it

Hello! I'm Gemma, a large language model created by the Gemma team at Google DeepMind. You can call me Gemma! 😊

Is there anything you'd like to ask me or talk about?

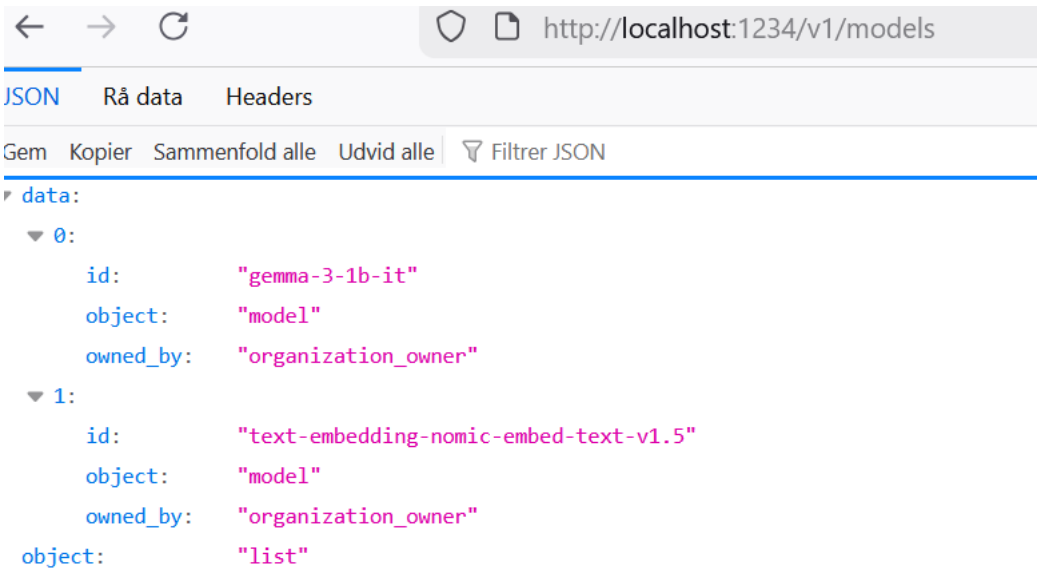
Exercise 2.

Now you must run your model as server.



Verify that it works in your browser:

http://localhost:1234/v1/models



Exercise 3.

Now you must call your LLM from a command prompt and verify that it works.

```
C:\>
C:\>curl http://localhost:1234/v1/chat/completions -H "Content-Type: application/json" -d "{\"messages\": [{\"role\": \"
user\", \"content\": \"Say hello!\"}]}"
{
  "id": "chatcmpl-59olfi8272be1guma4gzvb",
  "object": "chat.completion",
  "created": 1773679796,
  "model": "gemma-3-1b-it",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Hello there! How's your day going so far? 😊 \n\nLet me know if you want to chat about anything!",
        "tool_calls": []
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 12,
    "completion_tokens": 27,
    "total_tokens": 39
  },
  "stats": {},
  "system_fingerprint": "gemma-3-1b-it"
}
```

Exercise 4.

You now need to use the LLM running in LM studio in an agent workflow.

Things will probably be easier if you make a userfriendly interface to LM studio for experiments.

Can you get this (or similar) code to work (Make experiments, trial and error).

```
import gradio as gr
from openai import OpenAI

# 1. Point to your local LM Studio server
client = OpenAI(base_url="http://localhost:1234/v1", api_key="lm-studio")

def predict(message, history):
    # 2. Format history for the API
    messages = [{"role": "user", "content": message}]

    # 3. Call the local model
    response = client.chat.completions.create(
        model="local-model", # LM Studio uses whatever model is currently loaded
        messages=messages,
        temperature=0.7,
    )
```

```
    return response.choices[0].message.content

# 4. Launch the Gradio UI
gr.ChatInterface(predict).launch()
```

or:

```
import requests

url = "http://localhost:1234/v1/chat/completions"
headers = {"Content-Type": "application/json"}

data = {
    "messages": [
        {"role": "system", "content": "Du er en klog assistent."},
        {"role": "user", "content": "Hvem er onkel Anders?"}
    ],
    "temperature": 0.7
}

response = requests.post(url, headers=headers, json=data)
print(response.json()['choices'][0]['message']['content'])
```

Exercise 5.

Make a program where you chat with the LLM about books.

And when you agree about a certain book (you decide how this should be decided, based on output from the LLM) then you download the book (See code for this on Canvas).

Exercise 6.

In „Agents in Action“ a code challenge is described:

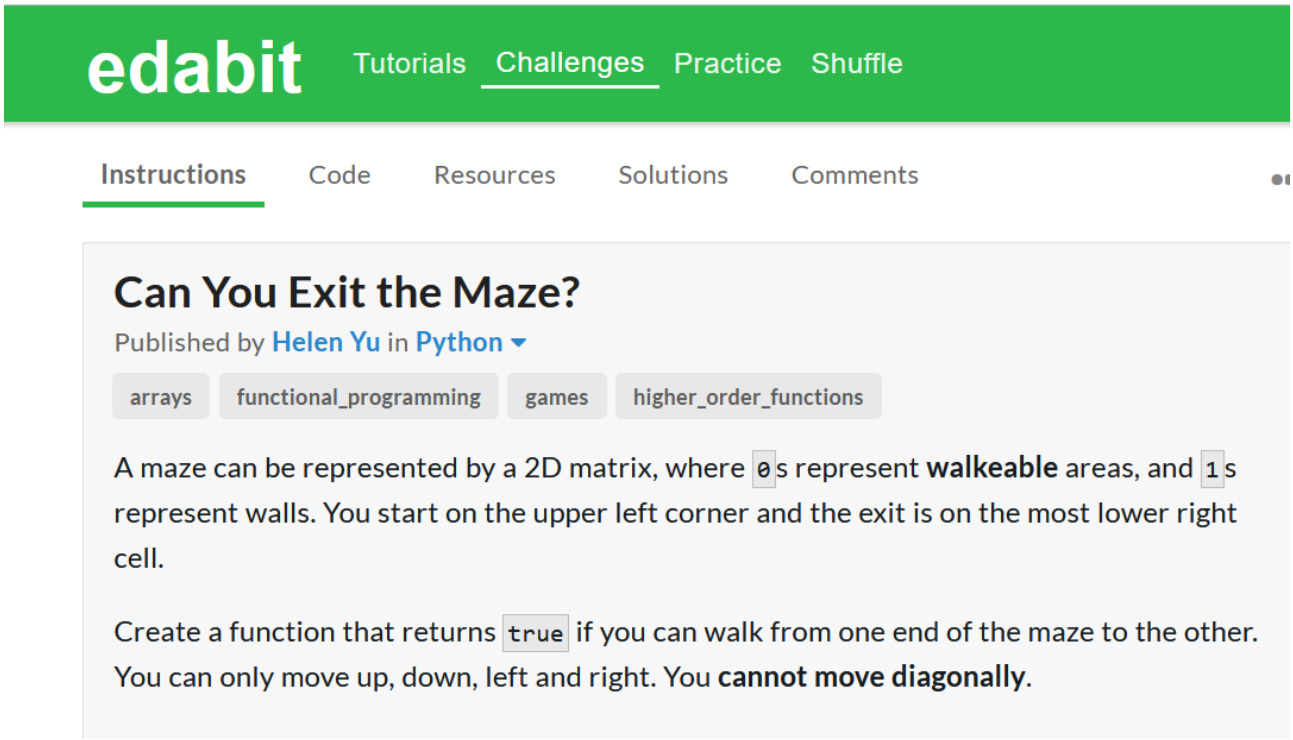
Coding challenges provide a good baseline for testing and evaluating agent and assistant systems. Challenges and benchmarks can quantify how well an agent or agentic system operates.

For this coding challenge, we're going a little further and looking at Python coding challenges from the Edabit site (https://edabit.com), which range in complexity from beginner to expert.

Lets try an expert challenge…

Actually, you might want to try out with a simpler challenge in ordert to get started. Solved this in a similar way as below, and then return to this *expert challenge*…

Ok on https://edabit.com lets look at the „Exit the Maze" challenge:



can_exit([
  [0, 1, 1, 1, 1, 1, 1],
  [0, 0, 1, 1, 0, 1, 1],
  [1, 0, 0, 0, 0, 1, 1],
  [1, 1, 1, 1, 0, 0, 1],
  [1, 1, 1, 1, 1, 0, 0]
]) ➞ true

can_exit([
  [0, 1, 1, 1, 1, 1, 1],
  [0, 0, 1, 0, 0, 1, 1],

```
  [1, 0, 0, 0, 0, 1, 1],
  [1, 1, 0, 1, 0, 0, 1],
  [1, 1, 0, 0, 1, 1, 1]
]) ➞ false

# This maze only has dead ends!

can_exit([
  [0, 1, 1, 1, 1, 0, 0],
  [0, 0, 0, 0, 1, 0, 0],
  [1, 1, 1, 0, 0, 0, 0],
  [1, 1, 1, 1, 1, 1, 0],
  [1, 1, 1, 1, 1, 1, 1]
]) ➞ false

# Exit only one block away, but unreachable!

can_exit([
  [0, 1, 1, 1, 1, 0, 0],
  [0, 0, 0, 0, 1, 0, 0],
  [1, 1, 1, 0, 0, 0, 0],
  [1, 0, 0, 0, 1, 1, 0],
  [1, 1, 1, 1, 1, 1, 0]
]) ➞ true
```

Great.

So, now we need our agent to come up with Python code that can solve this challenge…

What you need to do…

In a python program, you enter a maze, call an agent, that is LM Studio, that give you python code that solves the problem. You then call the Python code you have received from the LLM with your maze…

Problem solved – without coding it?

I.e.

Maybe you will get code like this from your LLM:

```
from collections import deque

def can_exit(maze):
    rows = len(maze)
    cols = len(maze[0])

    # Start og slut destination
    start = (0, 0)
    end = (rows - 1, cols - 1)

    # Hvis start eller slut er blokeret (1), er det umuligt
    if maze[0][0] == 1 or maze[rows-1][cols-1] == 1:
        return False

    queue = deque([start])
    visited = {start}

    while queue:
        r, c = queue.popleft()

        # Har vi nået udgangen?
        if (r, c) == end:
            return True

        # Tjek de 4 naboer (op, ned, venstre, højre)
        for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
            nr, nc = r + dr, c + dc

            # Tjek om naboen er inden for banen, er en vej (0) og ikke besøgt før
            if 0 <= nr < rows and 0 <= nc < cols and \
               maze[nr][nc] == 0 and (nr, nc) not in visited:

                visited.add((nr, nc))
                queue.append((nr, nc))
```

```
    return False
```

You can then call this code with your test cases:

```
# Test cases
maze1 = [
  [0, 1, 1, 1, 1, 1, 1],
  [0, 0, 1, 0, 0, 1, 1],
  [1, 0, 0, 0, 0, 1, 1],
  [1, 1, 0, 1, 0, 0, 1],
  [1, 1, 0, 0, 1, 1, 1]
]

maze2 = [
  [0, 1, 1, 1, 1, 0, 0],
  [0, 0, 0, 0, 1, 0, 0],
  [1, 1, 1, 0, 0, 0, 0],
  [1, 0, 0, 0, 1, 1, 0],
  [1, 1, 1, 1, 1, 1, 0]
]

print(can_exit(maze1)) # Output: False
print(can_exit(maze2)) # Output: True
```

The only thing missing is how to execute the code the LLM have given to you:

Try:

```
import runpy
runpy.run_path('min_kode.txt')
```

or

```
with open('min_kode.txt', 'r') as fil:
    kode = fil.read()
    exec(kode)
```